🧠 Day 09 — Objects: The Basics
أهم حاجة لازم تكون فاهمها كـ Front-End Developer

اليوم التاسع كان عن:

Objects
Object References & Copying
Garbage Collection
Object Methods + this
Constructor + new
Optional Chaining ?.
Symbol
Object → Primitive Conversion

لكن مش كلهم بنفس الأهمية في الشغل.

1️⃣ Objects ⭐⭐⭐⭐⭐

دي أهم حاجة في اليوم كله.

Object = بيانات مرتبطة ببعض
```
const user = {
  name: "Yousef",
  age: 22,
  role: "Frontend Developer"
};
```

تقدر تقرأ:

```
user.name
user.age
```

أو باستخدام Bracket:

```
user["name"];
الفرق المهم جدًا
user.name
```

هنا name مكتوبة حرفيًا.

لكن:

```
const key = "name";

user[key];
```

هنا JavaScript بتاخد قيمة key وهي "name".

⚠️ لذلك:

user.key

مش معناها user["name"].

هي بتدور على property اسمها "key".

```
Add / Update / Delete
user.city = "Cairo";   // Add
user.age = 23;         // Update
delete user.age;       // Delete
```

const مع Objects

دي لازم تكون ثابتة عندك:

```
const user = {
  name: "Yousef"
};

user.name = "Ahmed"; // ☑
```

لكن:

```
user = {}; // ✖
```

ليه؟

const يمنع تغيير الـ variable نفسه، لكنه لا يمنع تعديل الـ object الموجود داخله.

2️⃣ Nested Objects ⭐⭐⭐⭐⭐

هتستخدمها كتير جدًا مع APIs.

```
const user = {
  name: "Yousef",

  address: {
    city: "Cairo",
    country: "Egypt"
  }
};
```

الوصول:

```
user.address.city
```

وده مهم جدًا لأن الـ API responses غالبًا بتكون Nested.

3️⃣ for...in ⭐⭐⭐⭐

لو عايز تلف على Object: بتاعة Properties

```
for (const key in user) {
  console.log(key);
  console.log(user[key]);
}
```

ركز هنا:

```
user[key]
```

مش:

```
user.key
```

لأن key هنا variable.

4️⃣ Objects References ⭐⭐⭐⭐⭐

دي من أهم حاجات اليوم.

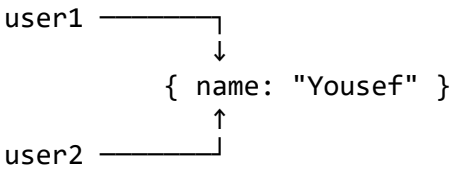
لما تعمل:

```
const user1 = {
  name: "Yousef"
};

const user2 = user1;
```

أنت مش عملت Object جديد.

الاتنين بيشاوروا على نفس الـ Object.



لذلك:

```
user2.name = "Ahmed";

console.log(user1.name);
```

النتيجة:

```
Ahmed
```

5️⃣ Shallow Copy ⭐⭐⭐⭐⭐

لو عايز تعمل Object جديد:

```
const user2 = { ...user1 };
```

دلوقتي الـ top-level مختلف.

يعني:

```
user2.name = "Ahmed";
```

مش هيغير:

```
user1.name
```

لكن خد بالك جدًا من الـ Nested Objects.

```
const user1 = {
  name: "Yousef",
  skills: {
    frontend: "React"
  }
};

const user2 = { ...user1 };
```

هنا:

```
user1 ──→ Object
           │
           └── skills ──→ Object A

user2 ──→ Object
           │
           └── skills ──→ Object A
```

يعني:

```
user2.skills.frontend = "JavaScript";
```

هيغير user1 كمان.

القاعدة:

Spread {...obj} يعمل Shallow Copy، مش Deep Copy.

6️⃣ Deep Copy ⭐⭐⭐⭐

لو عندك Object Nested وعايز نسخة مستقلة تمامًا:

```
const copy = structuredClone(user);
```

دلوقتي:

```
copy.skills.frontend = "JavaScript";
```

مش هيأثر على الأصل.

وده بالذات مهم جدًا لما نتعامل مع Nested Data.

7️⃣ Garbage Collection ⭐⭐⭐

مش مطلوب منك تحفظ تفاصيل Engine.

المهم تفهم الفكرة:

JavaScript بتشيل Objects اللي مش ممكن الوصول ليها من أي Root.

مثال:

```
let user = {
  name: "Yousef"
};

user = null;
```

لو مفيش أي reference تاني للـ Object، بقى unreachable وبالتالي Garbage Collector يقدر ينضفه.

أهم جملة:

```
Garbage Collection depends on Reachability, not simply whether you set something to null.
```

8️⃣ Object Methods ⭐⭐⭐⭐⭐

Function جوه Object تعتبر Method.

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log("Hello");
  }
};
```

وتستدعيها:

```
user.sayHi();
```

والـ preferred syntax:

```
sayHi() {}
```

بدل:

```
sayHi: function() {}
this ⋆⋆⋆⋆⋆⋆⋆
```

دي من أهم حاجات اليوم.

في:

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(this.name);
  }
};

user.sayHi();
```

هنا:

```
this === user
```

لأن الاستدعاء:

```
user.sayHi()
```

احفظ القاعدة دي:

this في الـ normal function method بتتحدد حسب طريقة استدعاء الـ function.

مثال مهم جدًا

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(this.name);
  }
};

const admin = {
  name: "Ahmed"
};

admin.sayHi = user.sayHi;

admin.sayHi();
```

النتيجة:

Ahmed

لأن وقت الاستدعاء:

```
admin.sayHi()
```

إذن:

```
this === admin
```

⑩ Arrow Function + this ⋆⋆⋆⋆

دي نقطة Interview مش لازم تستخدمها كل يوم، لكن لازم تفهمها.

Arrow function ملهاش this خاص بيها.

يعني:

```
const user = {
  name: "Yousef",

  sayHi: () => {
    console.log(this.name);
  }
};
```

❌ مش الطريقة الصحيحة لعمل method محتاجة this.

الأفضل:

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(this.name);
  }
};
```

لكن Arrow مفيدة جدًا داخل Method:

```
const user = {
  name: "Yousef",

  sayHi() {
    setTimeout(() => {
      console.log(this.name);
    }, 1000);
  }
};
```

الـ Arrow هنا بتاخد this من الـ surrounding function.

11 Constructor + new ⋆⋆⋆⋆

لما يكون عندك Objects كتير بنفس الـ structure:

```
const user1 = {
  name: "Yousef",
  age: 22
};

const user2 = {
  name: "Ahmed",
  age: 24
};
```

ممكن تستخدم Constructor:

```
function User(name, age) {
  this.name = name;
  this.age = age;
}
```

ثم:

```
const user1 = new User("Yousef", 22);
const user2 = new User("Ahmed", 24);
```

new بيعمل إيه؟

احفظها كده:

اعمل Object خلي → جديد this نفّذ الـ → يشاور عليه constructor → رجّع الـ Object.

يعني:

```
new User("Yousef", 22)
```

ينتج تقريبًا:

```
{
  name: "Yousef",
  age: 22
}
```

12 Optional Chaining ?. ⋆⋆⋆⋆⋆

مهمة جدًا في التعامل مع APIs.

لو عندك:

```
const user = {
  name: "Yousef"
```

};

وكتبت:

user.address.city

❌ ممكن تحصل:

TypeError

لأن address مش موجودة.

نستخدم:

user.address?.city

فتكون:

undefined

بدل Error.

Nested Optional Chaining

لو user نفسه ممكن يكون null:

user?.address?.city

دي بتحمي الاتنين:

user
 ↓
address
 ↓
city
الفرق:
user?.address.city

تحمي user فقط.

لكن:

user?.address?.city

تحمي user و address.

API Example ⭐⭐⭐⭐⭐

ده استخدام حقيقي جدًا:

```
const city = user?.address?.city;
```

ممكن يكون ناقص بيانات API response خصوصًا لما الـ.

13 Symbols ⭐⭐

دي مش أولوية كبيرة في شغلك اليومي.

المهم تفهم:

```
const id1 = Symbol("id");
const id2 = Symbol("id");

console.log(id1 === id2);
```

النتيجة:

```
false
```

كل Symbol() unique.

ممكن تستخدمه كـ Object key:

```
const id = Symbol("id");

const user = {
  name: "Yousef",
  [id]: 123
};
```

المطلوب منك هنا:

تفهم الفكرة، مش تحفظ كل تفاصيل Symbols.

14 Object → Primitive Conversion ⭐

دي أقل حاجة في اليوم من ناحية الاستخدام العملي.

الفكرة فقط:

لما JavaScript تحتاج تحوّل Object لـ primitive بيحصل بسبب عملية معينة، conversion.

مثلاً:

```
const obj = {
  toString() {
    return "2";
  }
};

console.log(obj * 2);
```

الناتج:

4

لأن Object اتحول لقيمة مناسبة للعملية.

المطلوب منك:

تفهم إن JavaScript أحيانًا تعمل implicit conversion للـ Objects.

مش مطلوب منك تحفظ تفاصيل Symbol.toPrimitive وترتيب valueOf() وtoString() دلوقتي.

لو هتحفظ 10 قواعد بس
1. Object = key/value collection.

2. user.name → property اسمها name.

3. user[key] → استخدم قيمة key كـ property.

4. const object يسمح بتعديل properties، لكن لا يسمح بإعادة إسناد object نفسه.

5. Objects are reference values.

6. {...obj} = Shallow Copy.

7. structuredClone(obj) = Deep Copy مناسب للحالات العامة المدعومة.

8. obj.method() → this = obj.

9. Arrow Function doesn't have its own this.

10. obj?.prop?.nested → Safe access لو جزء من الـ chain null/undefined.